

학습 결과서 — 딥러닝 파트

✧ Status **Finish**

담당 C · 데이터 전처리 v1.2 · 작성 2026-08-27

한 장 요약

우리가 알고 싶었던 것은 딱 하나입니다.

|"사람이 쓴 리뷰 글이, 그 사람이 게임을 그만둘지 맞히는 데 도움이 되는가?"

답은 이렇습니다.

질문	답
글에 신호가 있나?	있다. 글만 봐도 AUC 0.70
그럼 숫자에 글을 더하면 오르나?	거의 안 오른다. 0.813 → 0.810
왜 안 오르나?	글이 아는 걸 숫자가 이미 알고 있어서
그럼 글은 쓸모없나?	아니다. 처음 보는 게임에서는 글이 이긴다

한 줄로: 숫자는 그 게임 안에서 정확하고, 말은 게임을 건너뛰어도 통합니다.

1. 무엇을 재려고 했나

머신러닝 담당(B)은 **숫자만** 봅니다 — 플레이 시간, 보유 게임 수, 장르 같은 것들.

딥러닝 담당(C)은 **거기에 글을 더합니다.** 두 성능을 비교하면 글의 값어치가 나옵니다.

숫자만 → ?
숫자 + 글 → ?
—————
차이가 곧 "글의 효과"

왜 영어만 썼나

리뷰는 30개 언어로 쓰여 있습니다. 전부 넣으면 모델이 **글은 안 읽고 언어만 보는 지름길**을 탑니다 — "이건 러시아어네 → 그 게임이네 → 안 떠나겠네" 하는 식으로요.

그래서 가장 많은 영어(**67,112행**)만 썼습니다. 영어만 써도 이탈룰 차이가 0.9%p 뿐이라 데이터가 크게 치우치지 않습니다.

공정하게 비교하기

B가 낸 숫자(AUC 0.820)는 **13.9만 행 전체** 기준입니다. 우리는 6.7만 행만 씁니다.

행 수가 다르면 **글 덕분에 이긴 건지 데이터가 많아서 이긴 건지 알 수 없습니다.**

그래서 숫자만 쓰는 모델도 **영어 6.7만 행에서 처음부터 다시** 측정했습니다.

2. 두 가지 시험을 봅니다

시험	어떻게 나누나	무엇을 재나
랜덤 분할	6.7만 행을 섞어서 8:2	아는 게임에서의 실력
게임 단위 분할	게임 60개를 5조각. 학습에 없던 게임으로 시험	처음 보는 게임에서의 실력

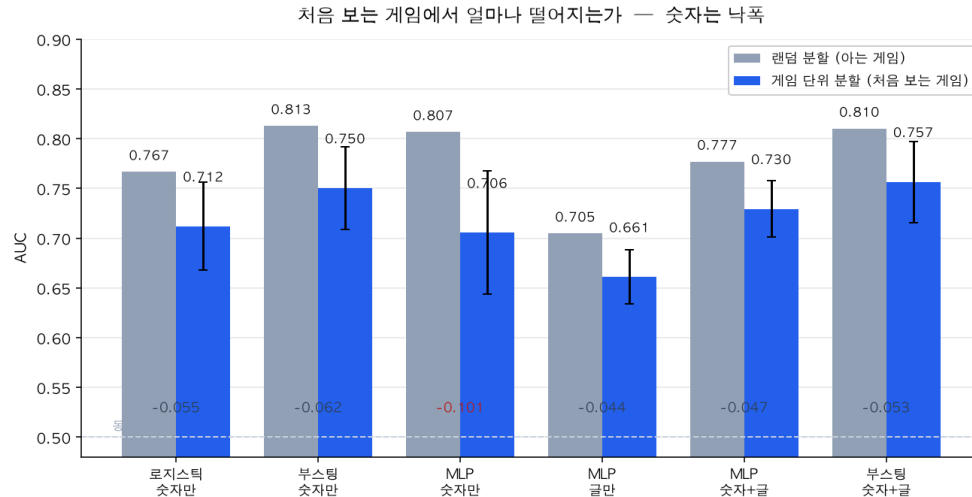
두 번째가 중요합니다. 게임마다 이탈률이 9%~89%로 천차만별이라, 모델이 "아 이거 그 게임이네" 하고 이름만 외워도 첫 번째 시험은 잘 봅니다.

두 점수의 차이 = 모델이 게임을 얼마나 외웠나. 이 문서에서는 그걸 **낙폭**이라고 부릅니다.

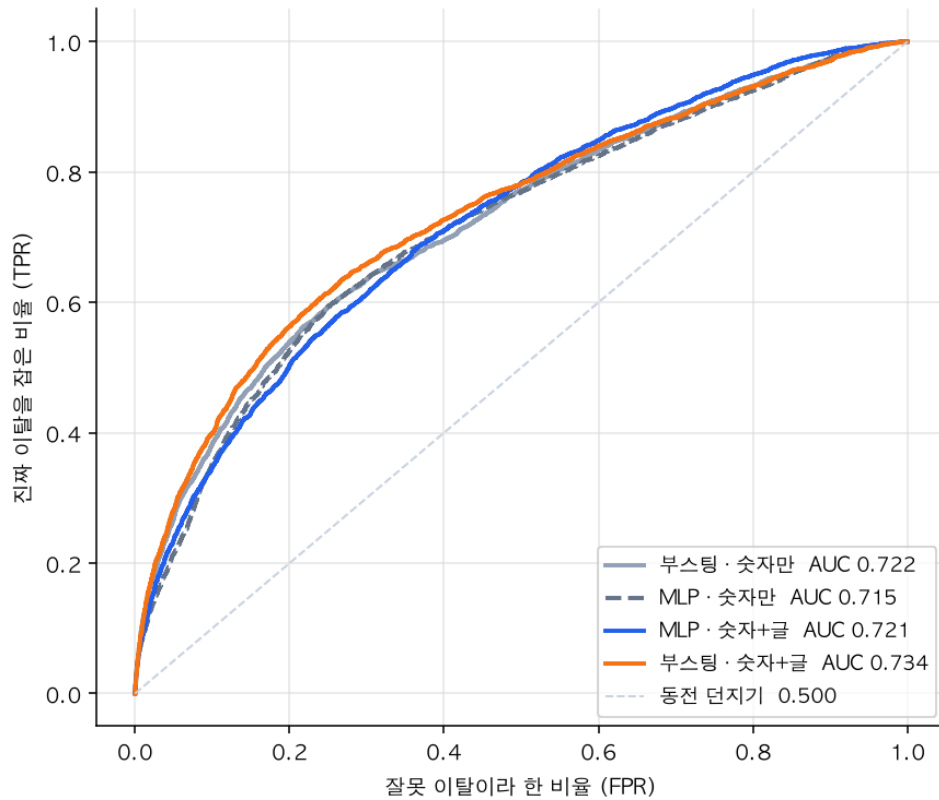
정확도(accuracy)는 쓰지 않습니다. 영어 데이터의 이탈률이 40.2%라, 아무것도 안 하고 전부 "안 떠난다"고만 찍어도 정확도 59.8%가 나옵니다. 대신 AUC를 씁니다 — 이탈한 사람과 안 떠난 사람을 하나씩 뽑았을 때 모델이 이탈자에게 더 높은 점수를 줄 확률입니다. 0.5는 동전 던지기, 1.0은 완벽.

3. 결과

모델	입력	랜덤 분할	게임 단위	낙폭
로지스틱	숫자만	0.767	0.712 ± 0.044	-0.055
부스팅	숫자만	0.813	0.750 ± 0.041	-0.063
MLP	숫자만	0.807	0.706 ± 0.062	-0.101
MLP	글만	0.705	0.661 ± 0.027	-0.044
MLP	숫자+글	0.777	0.730 ± 0.028	-0.048
부스팅	숫자+글	0.810	0.757 ± 0.041	-0.053



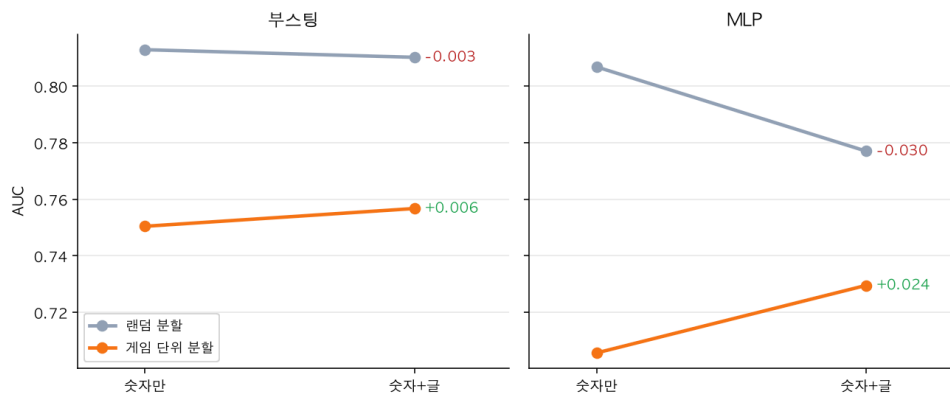
처음 보는 게임에서의 ROC 곡선
(게임 단위 분할 1조각)



4. 읽는 법 — 발견 세 가지

발견 ① 글만 봐도 맞힌다

글을 더하면 — 아는 게임에선 그대로, 처음 보는 게임에선 오른다



플레이 시간도, 보유 게임 수도 안 주고 **글만** 줬는데 AUC **0.705**가 나왔습니다. 동전 던지기(0.5)보다 훨씬 위입니다.

"재밌어요"와 "환불함"은 실제로 다른 신호입니다.

발견 ② 그런데 숫자에 더하면 안 오른다

부스팅 숫자만 0.813
부스팅 숫자+글 0.810 ← 오히려 아주 살짝 낮음

글이 아는 걸 숫자가 이미 알고 있기 때문입니다.

생각해보면 당연합니다. 30분 하고 "환불함"이라고 쓴 사람은 — 글을 안 읽어도 "30분"이라는 숫자가 이미 다 말해줍니다.

발견 ③ 처음 보는 게임에서는 글이 이긴다 ← 가장 중요

	랜덤 분할	게임 단위
부스팅 숫자만 → 숫자+글	0.813 → 0.810 (-0.003)	0.750 → 0.757 (+0.006)
MLP 숫자만 → 숫자+글	0.807 → 0.777 (-0.030)	0.706 → 0.730 (+0.024)

아는 게임에서는 글이 도움이 안 되는데, 처음 보는 게임에서는 도움이 됩니다.

낙폭을 보면 이유가 보입니다.

글만 -0.044 ← 가장 적게 떨어짐
숫자+글 -0.048
로지스틱 -0.055
부스팅 숫자만 -0.063
MLP 숫자만 -0.101 ← 가장 많이 떨어짐

왜 그럴까요.

숫자는 게임마다 뜻이 다릅니다. **Trailmakers** 에서 10시간은 이제 막 시작한 사람이고, **Half-Life: Blue Shift** 에서 10시간은 게임을 다 깬 사람입니다. 같은 "10시간"인데 뜻이 정반대입니다. 그래서 숫자로 배운 규칙은 새 게임으로 잘 안 옮겨갑니다.

말은 다릅니다. "환불함"은 어느 게임에서든 환불함입니다.

발표 한 줄

숫자는 그 게임 안에서 정확하고, 말은 게임을 건너뛰어도 통한다.

결다리 — 신경망이 게임을 제일 많이 외운다

MLP(숫자만)의 낙폭이 **-0.101**로 전 모델 중 최악입니다. 부스팅(-0.063)보다 1.6배 심하고, 조각별 편차도 ± 0.062 로 가장 큼니다.

아는 게임에서는 부스팅과 비슷해 보이지만(0.807 vs 0.813), 처음 보는 게임에서는 훨씬 크게 무너집니다.

5. 딥러닝이 부스팅에 졌습니다 — 괜찮습니다

부스팅 0.813 > MLP 0.807

표 형태 데이터에서 부스팅이 신경망을 이기는 것은 널리 알려진 결과입니다. 과제가 요구한 것은 "딥러닝이 이겨라"가 아니라 "둘을 같은 조건에서 비교하고 근거를 갖고 골라라" 입니다.

다만 글을 넣는 순간 이야기가 달라집니다. 글은 부스팅이든 신경망이든 넣을 수 있고, 넣었을 때 처음 보는 게임에서 둘 다 올랐습니다. 그게 이 파트의 성과입니다.

6. 최종 모델

고른 것 — MLP (숫자 + 글 64차원)

고른 기준 — 게임 단위 분할 성능. 실제 서비스에서는 학습에 없던 게임의 리뷰가 들어옵니다. 랜덤 분할 점수는 "이미 아는 게임에서의 실력"이라 실전과 다릅니다.

랜덤 분할	0.777
게임 단위	0.730
이탈 판정 기준	확률 0.335 이상

왜 0.5가 아니라 0.335인가: 기본값 0.5로 자르면 F1이 0.634인데, 0.335로 내리면 0.666으로 오릅니다. 이탈자가 40%뿐이라 기준을 조금 낮춰야 놓치는 사람이 줄어듭니다. 이 기준은 **학습에 쓰지 않은 데이터**에서 골랐습니다.

화면 담당에게 넘기는 것

models/dl_model.joblib	모델 (전처리까지 통째로 들어 있음)
models/dl_threshold.json	이탈 판정 기준 0.335
models/dl_feature_order.json	★ 열 순서
models/dl_meta.json	어떤 데이터로 만들었는지

dl_feature_order.json 이 가장 중요합니다. 모델은 열을 이름이 아니라 순서로 받습니다. 순서가 하나만 어긋나도 에러 없이 조용히 틀린 답이 나옵니다.

화면에서는 모델을 직접 부르지 말고 `src/predict.py` 의 함수 하나만 쓰면 됩니다.

```
from src.predict import predict_one
predict_one(review="...", playtime_at_review_min=25, ...)
# -> {"이탈확률": 0.968, "판정": "이탈"}
```

실제로 넣어본 예:

리뷰	플레이	결과
"Refunded after 30 minutes. Total waste of money."	25분	이탈 0.968
"300 hours in and still playing with friends every night."	300시간	잔존 0.192
"fun"	90분	이탈 0.887

세 번째가 재밌습니다. 글은 긍정적이는데(**fun**) **90분밖에 안 했다는 숫자가 이겼습니다**. 발견 ②를 눈으로 보여주는 사례입니다.

7. 한계 — 솔직하게

한계	내용
영어만	30개 언어 중 영어(48%)만 썼습니다. 다른 언어에서도 같은 결론일지는 확인하지 않았습니다

한계	내용
리뷰 쓴 사람만	애초에 리뷰를 쓴 사람만 데이터에 있습니다. 조용히 떠난 사람은 알 수 없습니다
게임 60개	다른 60개를 골랐으면 숫자가 달라질 수 있습니다
조각별 편차	게임 단위 분할은 조각마다 크게 흔들립니다($\pm 0.03 \sim 0.06$). 그래서 평균과 편차를 함께 적었습니다
<code>is_spike</code> 차이	학습 때는 "리뷰가 몰린 날인가"를 쓰는데, 화면에서는 알 수 없어 항상 0으로 넣습니다. 실제 화면 성능은 위 숫자보다 조금 낮을 수 있습니다
글 길이 잘림	임베딩 모델이 긴 글은 앞부분만 읽습니다. 3,000자 넘는 리뷰는 뒷부분이 반영되지 않습니다

8. 데이터 누수는 없었나

정답을 만드는 데 쓴 정보가 입력에 섞이면 성능이 비정상적으로 높게 나옵니다.

	AUC
누수 컬럼을 일부러 넣으면	0.998
우리 모델	0.777

전처리 담당이 금지 컬럼 9개를 미리 제거했고, 학습 직전에 **자동 검사**를 한 번 더 겁니다. 하나라도 섞이면 에러가 나서 학습이 시작되지 않습니다.

AUC가 0.95를 넘으면 기빠하기 전에 누수부터 의심합니다.

부록 — 재현 방법

```

uv sync                                # 환경 (맥·윈도우 동일)
uv run python -m src.preprocess        # 원본 -> dataset.csv
uv run python -m src.embed             # 리뷰 -> 임베딩 (약 1분)
uv run python -m src.train_dl          # 전 모델 학습 + 채점
uv run python -m src.figures           # 그림 3장
uv run python -m src.predict           # 예측 동작 확인

```

모든 결과는 `results/results.csv` 에 자동으로 쌓입니다.

난수 시드는 42로 고정했습니다.